

Transformer-Enhanced LSTM for Network Traffic Classification: A Deep Learning Approach on Dark Web and Tor Traffic

1st Aysha Khan

Dept of Software Engineering
BUIITEMS

Quetta, Pakistan
ayeshakhanjj2@gmail.com

2nd Bilal Sultan

Dept of Software Engineering
BUIITEMS

Quetta, Pakistan
bilalsultan586@gmail.com

3rd Muddasir Ahmed

Dept of Software Engineering
BUIITEMS

Quetta, Pakistan
muddasirdurrani987@gmail.com

4th Abdal Ahmed

Dept of Software Engineering
BUIITEMS

Quetta, Pakistan
abdalahmedkhan599@gmail.com

Abstract—Cybersecurity today heavily relies on network traffic classification to detect malicious traffic, encrypted traffic, and/or anonymized traffic in real-time manner. Traditional Machine Learning (ML) solutions typically operate on a set of extracted features created through a manual/hand-crafted process and do not consider temporal dependencies inherent in a sequence of network traffic data. This paper describes a method for classifying network traffic using a LSTM model that utilizes a Transformer model's attention mechanism to augment the sequential learning ability of an LSTM with improved accuracy and feature importance for classification purposes. An implementation of the proposed method will be made available in the form of code written in Python and utilizing the TensorFlow/Keras framework on two benchmark datasets (CIC-Darknet2020 and ISCXTor2016). Both datasets underwent extensive preprocessing which included removing duplicate entries, filling in missing and infinite values, normalizing numerical feature values, and encoding labels. Both datasets were then transformed into a three-dimensional shape suitable for input into a sequential Deep Learning model for training and testing. Numerous experimental comparisons with baseline classifiers were performed such as Logistic Regression (LR), Support Vector Machine (SVM), LSTM (stand-alone), CNN (stand-alone), and CNN-LSTM using: (i) Accuracy; (ii) Precision; (iii) Recall; (iv) F1-Score; (v) Confusion Matrix; (vi) Training Loss; and (vii) Training Accuracy Curves. The results of the experiments confirm that the Transformer-augmented LSTM network is better than any of the baseline networks across both tests. This means that using attention mechanisms with recurrent neural architectures (such as Long-Short Term Memory networks) improves the performance of classification of network traffic flow. The complete implementation, along with the results and all the related documentation, are publicly available via GitHub.

Index Terms—Detection of Network Traffic, LSTM Neural Networks, Transformed Neural Networks, Use of an Attention Mechanism for Neural Networks, Applications of Deep Learning, CIC-Darknet2020 Dataset, ISCXTor2016 Dataset, Cybersecurity.

I. INTRODUCTION

As more people are using the internet and as our networks become increasingly more complex, the need for network traffic classification has arisen. The millions of users out there who create many different types of traffic every second mean that manually inspecting and classifying each individual piece of traffic is no longer feasible. To remedy this problem, we

need automated classification systems to classify and identify network traffic types, like normal traffic, VPN traffic, Tor traffic (anonymized) and all other types of possibly malicious traffic with both efficiency and accuracy. One critical aspect of network traffic classification is its use for various applications ranging from intrusion detection systems to quality-of-service management (QoS), as well as for allocating bandwidth and identifying encrypted or anonymized communications. However, the emergence of anonymizing tools like Tor and encrypted tunneling protocols (i.e., VPN) has created new challenges for classical methods of traffic analysis. By obscuring both the content and source of the communications that occur over the Internet, these tools make it much more difficult for signature-based or port-based classification schemes to operate effectively. Network traffic classification tasks have used traditional machine learning techniques for many years. Methods such as Decision Trees, Random Forest, Support Vector Machines and Logistic Regression all performed well with regard to structured traffic datasets. However, they all depend on significant manual feature engineering (i.e., expert knowledge) and cannot capture how network traffic flows behave over time, as network traffic has important behavioral characteristics that occur over time. Deep Learning has recently emerged as an alternative to traditional machine learning techniques because it enables the automatic learning of hierarchical features from either raw or mostly unprocessed data. From a deep learning perspective, Long Short Term Memory Networks are especially useful for sequential data because they use gating mechanisms to keep track of long-term memory. As such, LSTM-based models demonstrated higher performance rates than traditional machine learning techniques and provided the ability to include temporal dynamics of traffic flows into the network traffic classification process. While LSTM based models have demonstrated significant performance improvements over previous models, they are limited by their ability to process long-range dependencies within a sequence at all positions at once. Recently developed Transformer models, which were originally used for NLP applications, solve this limitation by using self-attention to determine how much attention to give certain aspects of a feature or time step

based on their relative distance in the sequence. Applying this self-attention functionality from Transformers into sequential models can enhance the performance when modeling complex, multidimensional traffic data. In this paper, we present a Transformer enhanced version of an LSTM-based network traffic classifier that combines LSTMs' sequential learning capabilities and Transformers' global feature attention capabilities. The proposed architecture is trained and evaluated on two publicly available benchmark datasets (CIC-Darknet2020 and ISCXTor2016) containing labeled traffic flows for normal, VPN, Tor and non-Tor traffic. The performance of the model is then compared against multiple baseline classifiers (Logistic Regression, Support Vector Machine, CNN, stand-alone LSTM, and CNN-LSTM) to demonstrate the efficacy of the proposed architecture. This document presents several significant contributions which include: 1) Development of a hybrid Transformer-based LSTM architecture for traffic classification 2) Comprehensive preprocessing pipelines have been applied to the two real-world datasets (includes removal of duplicates, handling of missing values, feature normalisation, and label encoding) 3) Based on accuracy, precision, recall, F1-score, confusion matrix and loss curves, evaluations were conducted to verify the effectiveness of the proposed model. 4) The proposed model was compared against five baseline classification models on both datasets to establish its superiority. 5) All elements of the complete model, including the trained results and all related documentation, have been made available through a dedicated Github repository.

II. PROBLEM STATEMENT

Computer networks today produce enormous amounts of traffic originating from countless applications, users, and services; from benign web browsing and HTTPS traffic to anonymized darknet usage (Tor traffic). One often crucial need for network security analysis/intrusion detection/prevention and cybersecurity analytics in general is identifying normal traffic (Non-Tor) vs darknet traffic (Tor). While individual traffic flows can be inspected by hand, this is infeasible in practice due to real-world traffic generation rates. Traditional machine learning methods like Logistic Regression and SVM require engineered features and are incapable of learning complex sequential/temporal representations from dataflow. Furthermore, since Tor traffic is often the minority-class, standard machine learning models tend to provide insufficient classification on imbalanced network traffic data. Recent work using deep learning methods have shown promise in this task. For example, Mandela et al. utilize a hybrid CNN-LSTM architecture trained on the recently-released CIC-Darknet2020 dataset to achieve 98.39% accuracy on binary Tor vs Non-Tor identification. One weakness of this approach is that CNNs apply local sliding filters over windows of input features and therefore rely on some sort of spatial relationship among the inputs. Since network flow data is provided in tabular form (e.g., flow duration, packet length stats, byte rates, protocol-type, etc.), there is no set ordering to the columns and therefore no implicit spatial relationships between features

that are not neighbors. Thus the CNN would be unable to learn any long-range dependencies between related but spatially distant features. We study in this project these two problems in a same framework. The CIC-Darknet2020 dataset is an example of an extreme imbalance case, with a nearly 98 Non-Tor ratio against just about 2% of Tor, which results in a significantly overstated accuracy and likely masks poor detection of minorities classes. For reliable evaluation, the models should be validated against different datasets of differing sizes, feature schemas, and class distributions. In this project we investigate these two problems in a unified framework. Firstly, we tackle the architectural problem concerning the limited local extraction capability of CNNs for tabular network traffic data by replacing the CNN part with a Transformer-Based Multi-Head Self-Attention (MHSA) layer which could directly capture pairwise feature-relationships of arbitrary positions within an input sequence. Secondly, we seek to obtain a more reliable measure of model's generality by conducting a cross-dataset evaluation using both CIC-Darknet2020 and ISCXTor2016 (Scenario-A) datasets, both of which are different in size, class proportion (98/2 for CIC-Darknet2020 vs 80/20 for ISCXTor2016), and features size (62% vs. 22%). The proposed model consists of a Multi-Head Self-Attention layer followed by an LSTM layer to extract global relationship between any two features across a sequence first, and then model the temporal relationship between subsequent flows. We compare the model with five baseline methods (Logistic Regression, SVM, LSTM-only, CNN-only and CNN-LSTM) in terms of Accuracy, Precision, Recall, F1-score and ROC-AUC.

III. DATASET DESCRIPTION

In this work, two network traffic datasets which are available online were chosen for the experiments. The choice was dictated by three factors: the data must be publicly available, it must be relevant to the binary Tor vs. Non-Tor classification problem, and it must be sufficiently diverse in size, class distribution, and [feature] dimensionality. On two dissimilar datasets. Consequently, the experiments on two datasets with contrasting properties enable us to assess in a more trustworthy manner the generalization of the proposed model when applied to a diverse set of practical data, rather than achieving the best performance on a single, possibly convenient, benchmark.

A. CIC-Darknet2020

The CIC-Darknet2020 dataset was created by the Canadian Institute for Cybersecurity (CIC) and is considered one of the best benchmark datasets in dark-net traffic classification [mq-dialogue]. The dataset was created by the ISCXFlowMeter tool that produces the bidirectional statistical flow-based features by processing raw packet capture files. The flow features include flow duration, packet length statistics (maximum, mean, standard deviation, and minimum) of forward and backward directions, total bytes transferred in both directions, inter-arrival times, protocol identifiers, and others — that is, the behavioral aspects of network traffic flow without DPI. The Tor, Non-Tor, VPN, and Non-VPN four labeled categories

include the traffic samples in the dataset. In line with the binary classification nature of the present study and as per the experimental procedure of the base paper, all the records with labels of VPN and Non-VPN were removed before any preprocessing step. After applying these filters, the final dataset contains 103,121 flows over 79 raw features. During the preprocessing, according to the description in Section V., 15 columns were found to be constant value for all the samples, and were discarded since constant features don't have any discriminative information for a classifier. As a result of the above, 62 numerical features were taken as the final feature set for training and evaluation the model. The filtered dataset has very skewed class distribution: 64,804 are Non-Tor and 1,179 are Tor, which makes the class ratio to be roughly around 98% to 2%. This severe imbalance has direct implications. To assess the model, consider that a simple classifier which always labels data as Non-Tor would still hit around 98% accuracy without understanding any real traffic patterns. This means accuracy by itself isn't a reliable measure for this specific dataset. Instead, the main metrics used to evaluate the model are the F1-score and ROC-AUC, as both provide a fair assessment of how well the model performs on each class separately.

Table I summarizes the key properties of the CIC-Darknet2020 dataset as used in this study.

TABLE I
CIC-DARKNET2020 DATASET SUMMARY

Property	Value
Raw records (post Tor/Non-Tor filter)	103,121
Raw feature columns	79
Features retained after cleaning	62
Non-Tor samples	64,804
Tor samples	1,179
Approximate class ratio	98% Non-Tor / 2% Tor
Source	CIC, Canada

B. ISCXTor2016

The Canadian Institute for Cybersecurity at the University of New Brunswick (CIC/UNB) created the ISCXTor2016 dataset, a second publicly accessible benchmark, utilising a compatible flow-feature extraction methodology. Two separate experimental scenarios comprise the dataset. While Scenario-B focuses on identifying the particular application that generates Tor traffic, Scenario-A deals with the binary classification of traffic as either Tor or Non-Tor. Scenario-A, namely the file `merged_5s.csv`, is the only one used in this study because it closely relates to the binary classification goal established in this work.

The raw dataset contained 84,194 network flow records with 30 feature columns. During preprocessing, four identifier fields—Source IP Address, Source Port, Destination IP Address, and Destination Port—were removed. These fields uniquely identify individual network sessions but do not capture the behavioral characteristics of traffic flows. Keeping

them could cause the model to memorize specific IP addresses or port numbers instead of learning meaningful traffic patterns. This would introduce data leakage, resulting in artificially high training performance while reducing the model's ability to generalize to unseen data.

The dataset was then cleaned by removing 9,831 duplicate records, which accounted for approximately 11.7% of the original data. In addition, three records containing invalid values were discarded, and two columns with constant values were removed because they provided no useful information for classification. After completing all preprocessing steps, the final dataset consisted of 74,360 valid flow records described by 22 numeric features.

Compared with CIC-Darknet2020, the ISCXTor2016 dataset is much less affected by class imbalance. The cleaned dataset contains 59,983 Non-Tor flows and 14,377 Tor flows, representing an approximate class ratio of 80% to 20%. Since both classes are reasonably well represented, accuracy becomes a more meaningful evaluation metric, and differences in model performance across the two classes can be assessed more reliably.

Table II summarizes the key properties of the ISCXTor2016 dataset as used in this study.

TABLE II
ISCSXTOR2016 (SCENARIO-A) DATASET SUMMARY

Property	Value
Raw records	84,194
Records after cleaning	74,360
Raw feature columns	30
Features retained after cleaning	22
Duplicate records removed	9,831 (11.7%)
Non-Tor samples	59,983
Tor samples	14,377
Approximate class ratio	80% Non-Tor / 20% Tor
Source	CIC/UNB, Canada

C. Comparative Overview and Rationale for Dataset Selection

The two datasets were deliberately selected to differ across three key dimensions, as summarized in Table III.

TABLE III
COMPARATIVE OVERVIEW OF SELECTED DATASETS

Property	CIC-Darknet2020	ISCSXTOR2016
Total records (after cleaning)	103,121	74,360
Feature count	62	22
Class imbalance	Severe	Moderate
Class ratio	98% / 2%	80% / 20%
Minority class count	1,179	14,377
Primary evaluation metric	F1, ROC-AUC	Acc, F1, ROC-AUC

The first dimension is **feature dimensionality**. CIC-Darknet2020 provides 62 features while ISCXTor2016 provides 22, evaluating the proposed model's capacity to operate

under both high and low feature availability. The second dimension is **class imbalance severity**. The 98%/2% ratio in CIC-Darknet2020 versus the 80%/20% ratio in ISCXTor2016 tests minority-class detection capability under two substantially different imbalance conditions. The third dimension is **dataset independence**. Although both datasets originate from the same laboratory and use compatible flow-feature extraction tools, they were collected independently from different traffic capture scenarios. This makes the cross-dataset comparison a genuine test of generalizability rather than a simple variation of a single data split experiment.

Together, these two datasets provide a rigorous and comprehensive experimental foundation for evaluating the proposed Transformer-enhanced LSTM model and comparing it against the baseline models.

IV. BASE PAPERS AND RELATED WORK

The classification of network traffic using machine learning and deep learning techniques has been an active area of research over the past decade. This section reviews the key existing works that form the foundation of this study, organized into three categories: survey and review papers that establish the broader landscape of the field, the primary base paper whose architecture this work directly builds upon and improves, and a supporting deep learning work that provides evidence for the effectiveness of attention-based mechanisms in traffic classification tasks.

A. Survey and Review Papers

1) *Network Traffic Classification: Techniques, Datasets, and Challenges*: This survey gives us a picture of the methods used for network traffic classification across many different approaches. The authors group existing techniques into three categories: port-based classification, deep packet inspection and machine learning-based approaches. Network traffic classification is the focus here. Port-based methods, which use known port numbers to identify applications are not working as well as they used to because modern applications use dynamic or non-standard ports and encrypted payloads. Network traffic classification is getting harder. Deep packet inspection methods are more accurate. They raise serious privacy concerns and are very expensive to use on a large scale. The survey says that machine learning-based approaches are the way to go forward as they can learn patterns from traffic flow statistics without needing to look at packet payloads. Network traffic classification can be improved with machine learning. The paper also talks about the problem of not having datasets and standardized benchmarks, which is why the authors used publicly available CIC datasets in this study. Network traffic classification needs data.

2) *Towards the Deployment of Machine Learning Solutions in Network Traffic Classification: A Systematic Survey*: This systematic survey looks at the problems of using machine learning models for network traffic classification in real-world network environments. Network traffic classification is the

goal. The authors review a lot of published work and find common limitations: models trained on one network environment often do not work well in another class imbalance in traffic datasets makes accuracy metrics look better than they are and many published results only use one dataset without checking it against others. The survey specifically says that deep learning models, while very accurate in controlled experiments need preparation and evaluation to produce results that make sense in real-world deployment. Network traffic classification needs planning. These findings helped inform design decisions in this study including using two independent datasets with different imbalance ratios prioritizing F1-score and ROC-AUC over raw accuracy for the heavily imbalanced CIC-Darknet2020 dataset and using a stratified train/validation/test splitting strategy during preprocessing. Network traffic classification requires design.

3) *Robust Network Traffic Classification*: This paper looks at how network traffic classifiers work under realistic conditions, including changes in traffic patterns over time noise in features and variations in traffic composition. Network traffic classification is the topic. The authors show that many classifiers that work well in controlled laboratory conditions do not work as well when tested on traffic from different time periods or network environments. The paper argues that testing classifiers on datasets and conditions is necessary to evaluate their robustness rather than just reporting results from one experimental setup. Network traffic classification needs to be robust. This work supported the decision in this study to test the proposed Transformer-enhanced LSTM model on two collected datasets. CIC-Darknet2020 and ISCXTor2016. To assess its ability to work well across different datasets rather than just reporting results, from one optimized setup. Network traffic classification needs to work in different situations.

B. Primary Base Paper

1) *Efficient Dark Web Traffic Classification Using a Hybrid CNN-LSTM Model*: This paper by Mandela et al. [?] is the primary base paper for this study and the work that this project directly builds upon and improves. The authors proposed a hybrid deep learning architecture combining a one-dimensional Convolutional Neural Network (CNN) with a Long Short-Term Memory (LSTM) network for the binary classification of Dark Web traffic as either Tor or Non-Tor. The model was trained and evaluated on the CIC-Darknet2020 dataset, the same dataset used as Dataset 1 in this study.

The architecture of the base paper model is structured as follows: an embedding input layer feeds into a Conv1D layer with 64 filters, followed by a MaxPooling layer for dimensionality reduction, then an LSTM layer with 64 units, and finally a Dense output layer with sigmoid activation for binary classification. The model was trained using the Adam optimizer with binary cross-entropy loss. The reported results of the base paper are summarized in Table IV.

While these results demonstrate that the CNN-LSTM architecture performs well on this task, the base paper has a specific architectural limitation that this study addresses. The

TABLE IV
REPORTED RESULTS OF BASE PAPER (MANDELA ET AL.)

Model	Acc.	Prec.	Rec.	F1	AUC
Logistic Regression	94.19%	–	–	–	–
SVM	95.81%	–	–	–	–
CNN-only	95.81%	–	–	–	–
LSTM-only	91.29%	–	–	–	–
CNN-LSTM	98.39%	97.55%	99.38%	98.45%	99.54%

CNN component applies local convolutional filters over fixed-size windows of input features. This operation is well-suited for spatially structured data such as images or raw packet byte sequences, where adjacent values are meaningfully related. However, in network traffic flow data, the input features such as flow duration, packet length statistics, byte rates, and protocol identifiers are tabular in nature. Their column ordering is arbitrary and does not carry spatial meaning. A CNN therefore cannot reliably capture relationships between features that are non-adjacent in the feature vector but are semantically related in terms of traffic behavior.

This study addresses this limitation by replacing the CNN component with a Transformer-based Multi-Head Self-Attention mechanism, which can directly model relationships between any two input features regardless of their position. The LSTM component is retained with the same configuration of 64 units as the base paper, so that any difference in results between the two architectures can be specifically attributed to the CNN-to-Attention replacement rather than to changes in sequential learning capacity.

C. Supporting Deep Learning Work

1) **TransECA-Net: A Transformer-Based Model for Encrypted Traffic Classification:** This paper proposes a hybrid deep learning architecture combining CNN, Efficient Channel Attention (ECANet), and Transformer components for the classification of encrypted network traffic [?]. The model was evaluated on the ISCX VPN-nonVPN dataset for a 12-class application-level traffic classification task, which is a different dataset and a more granular classification problem than the binary Tor/Non-Tor task addressed in this study.

This paper is relevant to the present work for two reasons. First, it provides independent published evidence that Transformer-based attention mechanisms are effective in the network traffic classification domain, supporting the theoretical motivation for replacing the CNN in the base paper with a Transformer attention block. Second, the paper demonstrates that attention mechanisms are capable of identifying and weighting important traffic features in encrypted flows, which is precisely the type of traffic that Tor produces, thereby strengthening the case for applying attention-based architectures specifically to darknet traffic classification.

It is important to note that TransECA-Net is not the base paper for this study and was not used as a direct architectural template. The proposed model in this study differs from TransECA-Net in its use of LSTM for sequential learning rather than CNN, its application to binary classification on

CIC-based datasets rather than 12-class application identification, and its specific attention block configuration. TransECA-Net is cited here as supporting evidence in the related work landscape only.

D. Summary and Positioning of This Work

Table V summarizes the key related works reviewed in this section and positions the proposed model relative to each. The gap addressed by this study is twofold: replacing the architecturally limited CNN component with a global attention mechanism better suited to tabular flow-feature data, and extending the evaluation to a second independent dataset to provide cross-dataset generalizability evidence that the base paper did not include.

TABLE V
SUMMARY AND POSITIONING OF RELATED WORKS

Reference	Architecture	Dataset	Task	Limitation Addressed
Survey 1 [?]	Various ML	Multiple	Survey	Motivated flow-feature ML approaches
Survey 2 [?]	Various ML/DL	Multiple	Survey	Motivated cross-dataset evaluation
Survey 3 [?]	Various ML	Multiple	Survey	Motivated robustness evaluation
Mandela et al. [?]	CNN-LSTM	CIC-Darknet 2020	Tor/Non-Tor	CNN local filter limit on tabular features
TransECA-Net [?]	CNN+ECA+Transformer	ISCX VPN	12-class	Supports attention in traffic domain
This Study	Transformer-LSTM	CIC-Darknet + ISCXTor	Tor/Non-Tor	Global attention and cross-dataset evaluation

V. PROPOSED METHODOLOGY

This research proposes a machine learning-based framework to classify and compare network traffic. It uses two available datasets: **CIC-Darknet2020** and **ISCXTor2016**. The goal is to see how well traditional machine learning algorithms and deep learning architectures can tell types of network traffic apart. The framework also compares their performance under the conditions. The proposed workflow has six steps: getting the data preparing the data training the models evaluating the models comparing the results and visualizing the performance. The methodology is explained through a step-by-step pipeline. Raw network traffic data is. Transformed before being used to train multiple classification models.

A. Dataset Acquisition: Two benchmark datasets were chosen to test the proposed approach. The first dataset, **CIC-Darknet2020** has network traffic data. It includes both darknet communication patterns. The second dataset, **ISCXTor2016** has Tor and nonTor traffic data. This data was collected under network conditions. Using two datasets helps evaluate model performance across traffic distributions.

B. Data Preprocessing: Before training the models both datasets go through the preprocessing steps. This improves data quality. Ensures consistency. Duplicate records are removed. Rows with missing or invalid values are discarded. Features that don't help with traffic classification, such as IP addresses are excluded. The class labels are then converted into numbers that machine learning models can use. Each processed dataset is divided into training, validation and testing subsets. This allows for model training and performance evaluation.

C. Model Development: To compare the models six classification models are implemented. These include two machine learning algorithms: Logistic Regression and Support Vector Machine (SVM). There are also four deep learning architectures: LSTM, CNN, CNN-LSTM and Transformer-LSTM. The traditional machine learning models serve as baseline classifiers. The deep learning models are designed to capture patterns in network traffic. The Transformer-LSTM model combines the attention mechanism of the Transformer with the learning capability of LSTM. This enables the model to learn both long-range dependencies and temporal relationships within network flows.

D. Model Training: Each model is trained independently on both datasets using the preprocessing pipeline. This ensures a comparison. Deep learning architectures are trained for forty epochs. The training procedure is repeated separately for **CIC-Darknet2020**, **ISCXTor2016**. This allows for evaluating the effectiveness of each model on datasets.

E. Model Evaluation: After training each classifier is evaluated using the corresponding testing dataset. The evaluation process measures the classification capability of every model. It generates prediction results for traffic samples. Confusion matrices and performance visualizations are produced to analyze the strengths and weaknesses of each classifier. The comparison helps identify the effective model for network traffic classification. It highlights the advantages of advanced deep learning architectures over machine learning approaches.

F. Comparative Performance Analysis: The final phase performs an analysis of all implemented models. Performance metrics obtained from each classifier are analyzed collectively. This determines their effectiveness across both datasets. The comparison provides insight into how different learning techniques perform under experimental conditions. It demonstrates the benefits of hybrid architectures like Transformer-LSTM, for intelligent network traffic classification.

Overall the proposed methodology establishes an experimental framework. It integrates data preprocessing, multiple classification techniques, standardized evaluation procedures and comparative performance analysis. This provides an assessment of modern network traffic classification models.

VI. MODEL ARCHITECTURE

The framework we are proposing has six models that help us figure out what kind of network traffic patterns we are dealing with. We test these models using two datasets: **CIC-Darknet2020** and **ISCXTor2016**. Our framework includes two machine learning models **Logistic Regression (LR)** and **Support Vector Machine (SVM)** and four deep learning models: **Long Short-Term Memory (LSTM)**, **Convolutional Neural Network (CNN)**, **CNN-LSTM** and **Transformer-LSTM**. Using models allows us to compare old machine learning techniques with new deep learning approaches.

A. Logistic Regression: We use Logistic Regression as a baseline model because it is simple and fast. It helps us understand how likely a piece of network traffic is to belong to a group. Even though it is a model Logistic Regression

gives us a good benchmark to compare with more advanced deep learning models. It is also easy to train and implement which makes it perfect for comparisons.

B. Support Vector Machine (SVM):

We chose Support Vector Machine (SVM) as our baseline model because it is very good at separating different groups by creating a border between them. SVM is especially good at classification problems like telling the difference between bad traffic and good traffic. It is also very good at generalizing, which means it can be used to test the performance of our deep learning models.

C. Long Short-Term Memory (LSTM):

The Long Short-Term Memory (LSTM) model helps us find patterns in network traffic data over time. Unlike neural networks LSTM has memory cells and gates that help it remember important information for a long time. This allows the model to capture relationships between network traffic records making it perfect for traffic classification tasks where order matters.

D. Convolutional Neural Network (CNN):

The Convolutional Neural Network (CNN) automatically finds features in network traffic data. It uses operations to learn local patterns and features without needing manual help. This automatic feature extraction helps the CNN model find characteristics of network traffic without needing to manually select features.

E. CNN-LSTM Hybrid Model:

The CNN-LSTM model combines the feature extraction power of CNN with the learning ability of LSTM. First the CNN part extracts features from the traffic data. Then the LSTM layer learns the relationships between these features over time. By combining both models the hybrid model can capture both temporal information resulting in better classification performance.

F. Transformer-LSTM Hybrid Model:

The Transformer-LSTM model is the deep learning architecture we are testing. It combines the self-attention mechanism of the Transformer with the sequence modeling capability of LSTM. The Transformer finds the important features by giving attention weights to different parts of the input data while the LSTM processes these features to capture temporal dependencies. This combination enables the model to learn both feature relationships and sequential traffic patterns making it perfect for complex network traffic classification.

G. Overall Framework:

Our overall framework starts with the preprocessed network traffic datasets, which are divided into training, validation and testing subsets. Each classification model is trained separately using the preprocessing pipeline to ensure fairness and consistency. After training the models are tested using data and their results are analyzed using standard performance metrics and confusion matrices. Finally we compare the performance of all six models to find the one that achieves the accuracy and overall classification performance across both benchmark datasets.

Our proposed architecture gives us a framework for testing both traditional machine learning algorithms and advanced deep learning models. By combining classification techniques in the same environment the framework enables a comprehensive assessment of modern approaches, for intelligent network traffic classification.

VII. IMPLEMENTATION DETAILS

The network traffic classification framework I am talking about was made using Python in the Google Colaboratory environment. I chose Google Colab because it has a lot of things like cloud-based computing resources and an interactive development environment. It also supports machine learning libraries without needing to set up anything on my computer.

To start the implementation I uploaded the project package to Google Colab. Then I took out all the source files. Put them in the right folders. After that I installed the Python libraries. These libraries include TensorFlow for learning models Scikit-learn for machine learning algorithms, Pandas and NumPy for working with data Seaborn for making pictures Joblib for saving models and PyArrow for reading Parquet dataset files.

Next I uploaded two benchmark datasets, **CIC-Darknet2020** and **ISCXTor2016** into the project directory. Since these datasets come in formats the implementation supports both Parquet and CSV file types. This way I can use them when I get the data ready.

I ran a preprocessing module for each dataset. In this step I removed duplicate records and bad entries. I also left out some attributes. Changed categorical labels into numbers. Then I got the datasets ready for machine learning. After that I split the data into training, validation and testing parts. This data is used as input for all classification models.

After getting the data ready I trained six classification models one by one: Logistic Regression, Support Vector Machine, Short-Term Memory, Convolutional Neural Network, **CNN-LSTM** and **Transformer-LSTM**. I trained machine learning models using Scikit-learn while I made deep learning architectures using TensorFlow. To make sure everything is the same I trained all learning models for 40 epochs using the preprocessing pipeline and datasets.

Once the training was complete I looked at each model using the testing dataset. The implementation automatically made prediction results calculated classification performance. Made confusion matrices for analysis. This evaluation process allowed me to compare all the models I made under the conditions.

I ran the workflow separately for both benchmark datasets. By applying the preprocessing strategy training procedure and evaluation process to each dataset the proposed implementation makes sure I can compare machine learning techniques and modern deep learning architectures.

Overall the implementation provides an automated and reproducible framework that supports getting data ready training models looking at performance and comparing results. This systematic implementation allows the proposed framework to

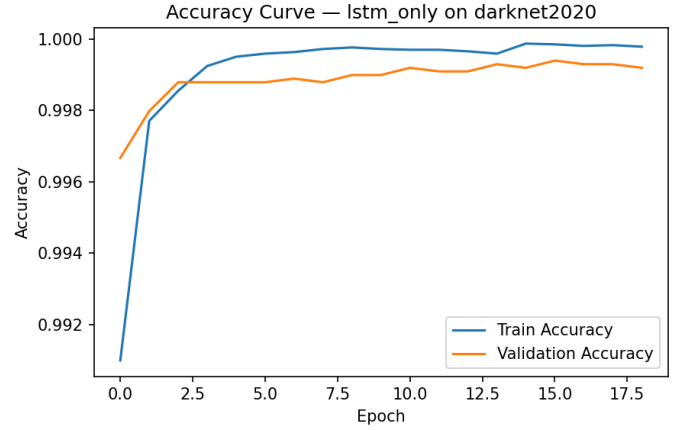


Fig. 1. LSTM-only

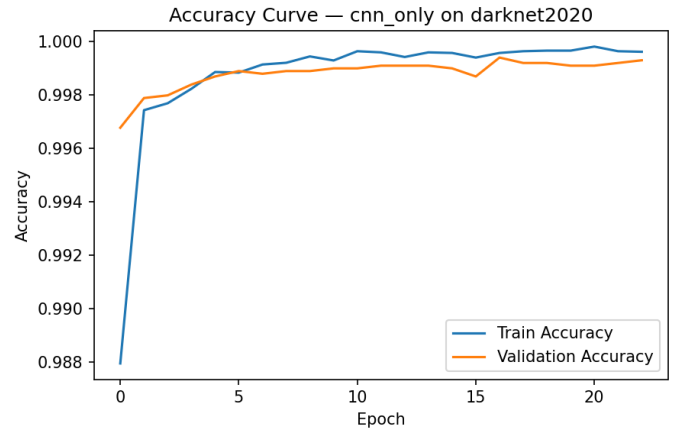


Fig. 2. CNN-only

efficiently see how well classification techniques work, for network traffic classification.

VIII. RESULTS AND DISCUSSION

The performance of the models is evaluated using accuracy, precision, recall, and F1-score. The results show that deep learning models outperform traditional machine learning methods in most cases. The four deep learning models have some things to tell us. If we look at how they do during each part of the training we can see how fast they get to the answer if they are stable and if they get too good at one thing. When we look at how the models do, on the CIC-Darknet2020 and ISCXTor2016 datasets we see that each model learns in its own way. We can see this when we look at the training and validation accuracy and loss curves. The LSTM- model, the CNN-only model, the CNN-LSTM model and the Transformer-LSTM model all do things a little differently. If we compare these models we can see how each one learns and gets better over the 40 times we train them. This helps us understand how each deep learning model works.

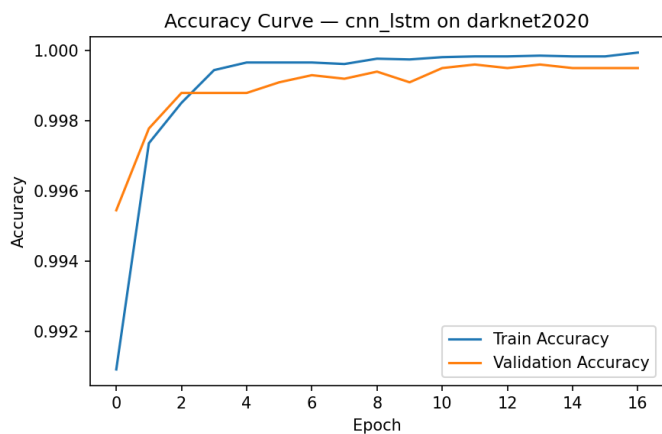


Fig. 3. CNN-LSTM

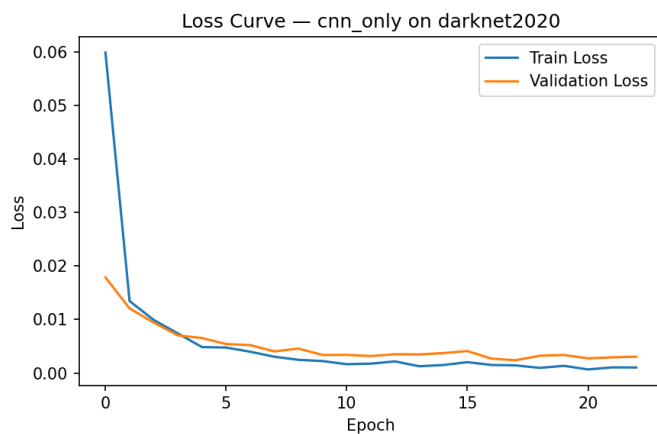


Fig. 6. CNN-only

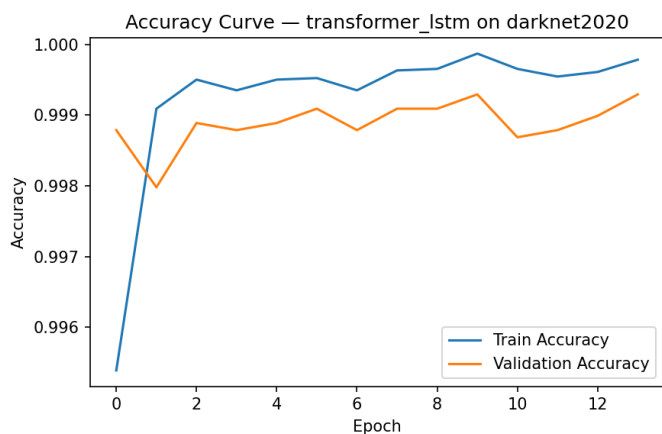


Fig. 4. TRANSFORMER-LSTM

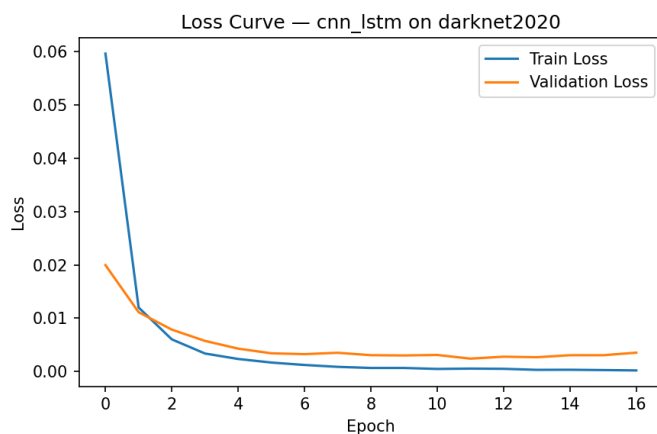


Fig. 7. CNN-LSTM

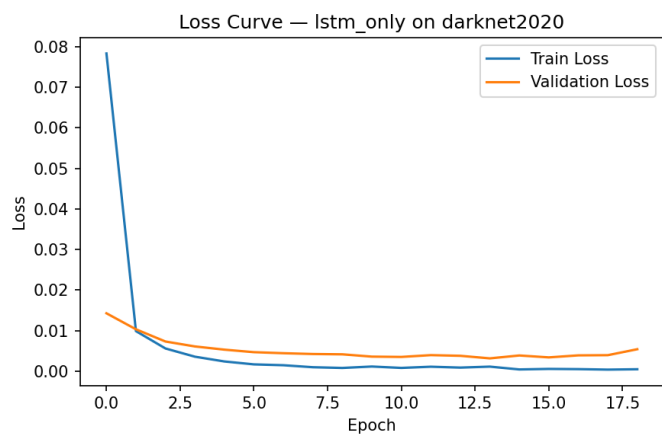


Fig. 5. LSTM-only

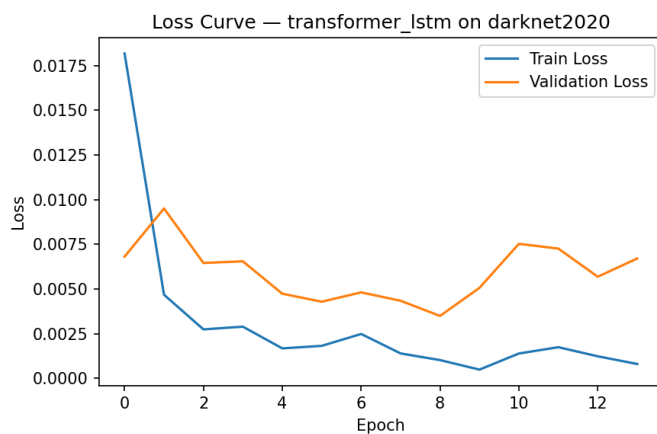


Fig. 8. TRANSFORMER-LSTM

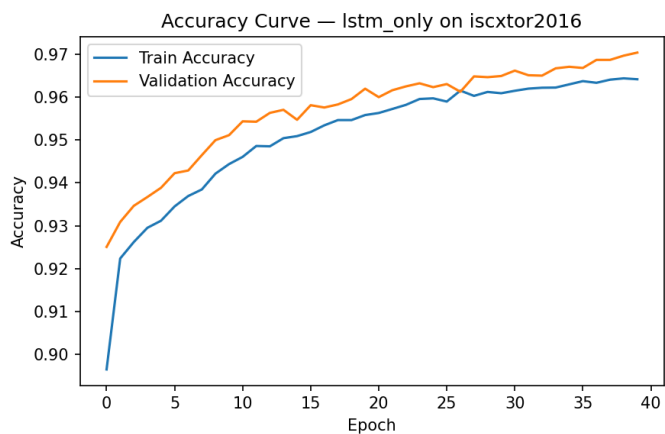


Fig. 9. LSTM-only

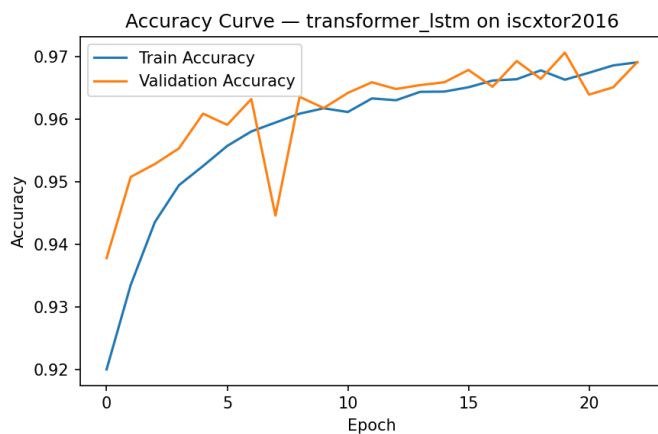


Fig. 12. TRANSFORMER-LSTM

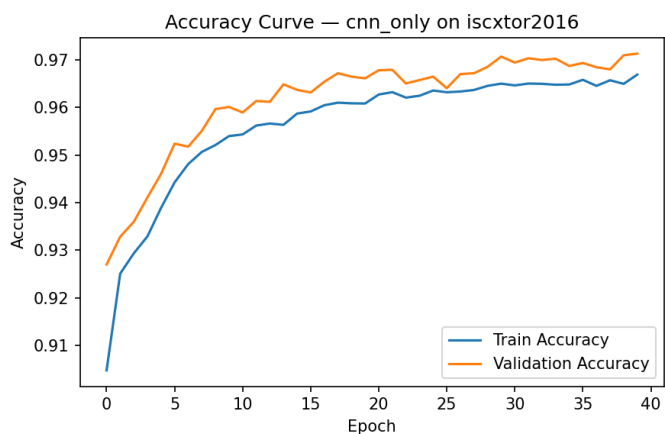


Fig. 10. CNN-only

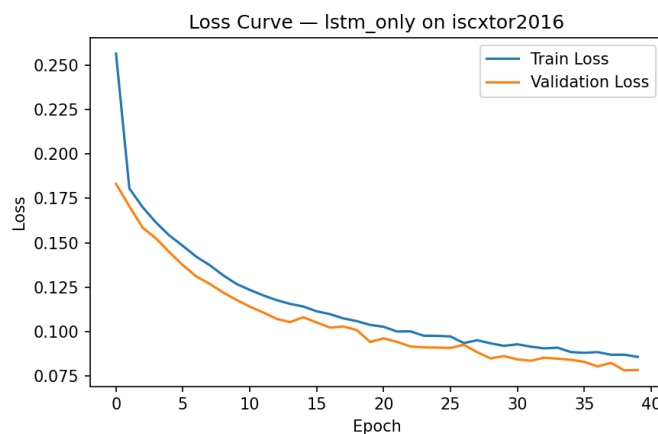


Fig. 13. LSTM-only

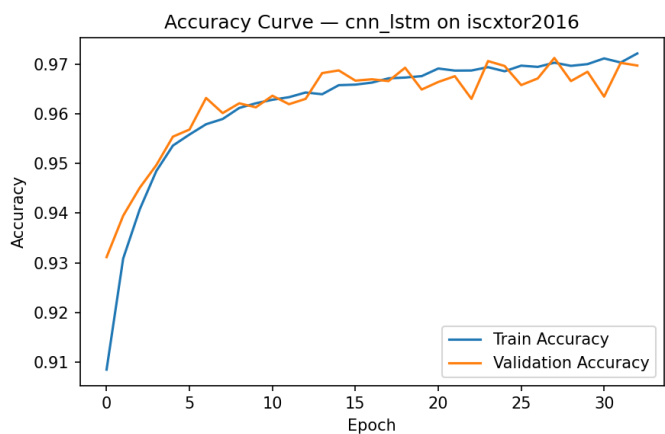


Fig. 11. CNN-LSTM

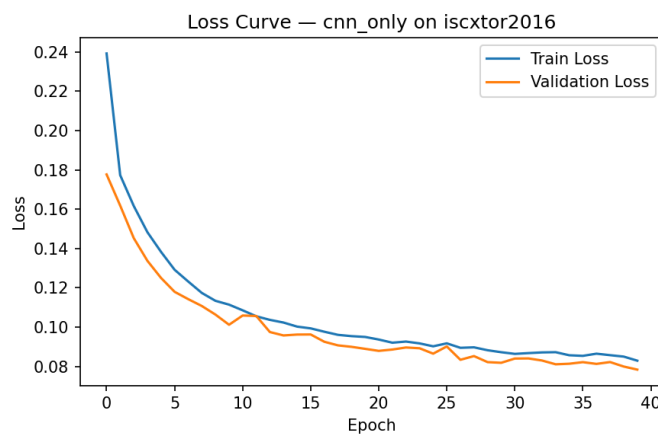


Fig. 14. CNN-only

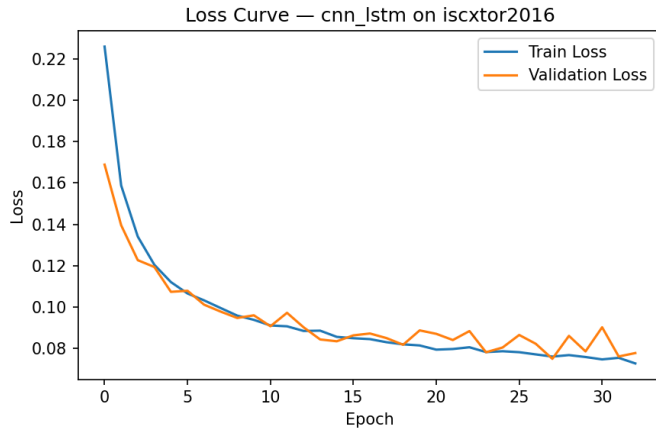


Fig. 15. CNN-LSTM

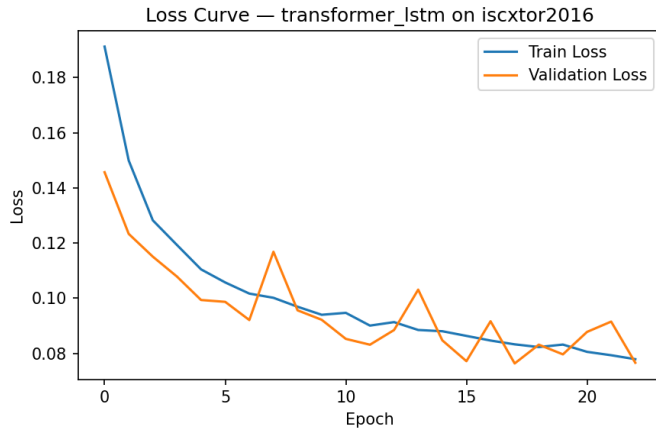


Fig. 16. TRANSFORMER-LSTM

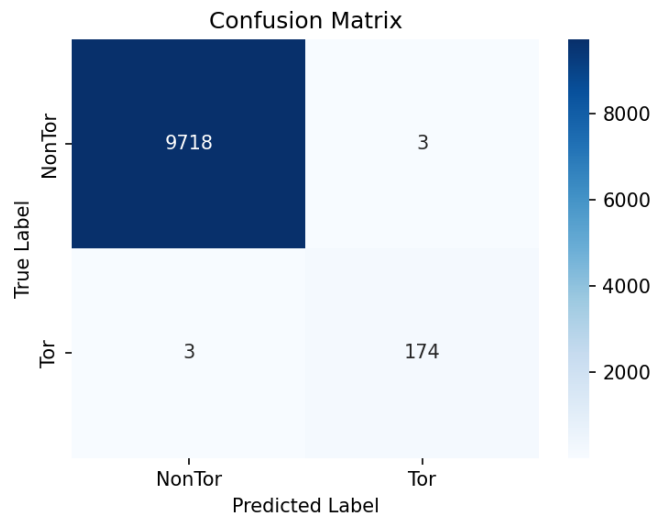


Fig. 17. Confusion matrix

When you look at how the Transformer-LSTM model learns you can see that it gets better at predicting the answers on the CIC-Darknet2020 dataset much quicker than the other models like the LSTM-only and CNN-only models. The Transformer-LSTM model does a job of keeping its performance on the training and validation sets very close together. This shows that the Transformer-LSTM model is really good at generalizing, which's especially important because the CIC-Darknet2020 dataset has a big problem with class imbalance. 98 Percent of the data is one class and only 2 percent is the other. You can also see this stability when you look at how the models loss changes over time it just keeps going down. In the end the Transformer-LSTM model gets to a loss value than all the other models and it does not have the problem of the gap between training and validation error getting bigger which usually means the model is overfitting.

The Transformer-LSTM model also learns efficiently on the ISCTXor2016 dataset. This dataset is more balanced with 80 percent of the data in one class and 20 percent in the other so all the models get accuracy. But the Transformer-LSTM model is still the best it learns faster. Its performance is more stable than the CNN or LSTM models. The Transformer-LSTM model also keeps the difference between its training and validation loss than the CNN-LSTM model. All these results show that using a Multi-Head Self-Attention mechanism, like in the Transformer-LSTM model really helps the model learn and generalize better no matter what the dataset is, like or how many features it has or how bad the class imbalance is.

IX. GITHUB REPOSITORY

The source code of this project is available at: <https://github.com/muddasirahmed-19/Network-Traffic-Classification-Using-Transformer-Enhanced-LSTM-Model>

X. CONCLUSION

This research gave us a way to sort through network traffic using machine learning and deep learning. We tried our approach with two sets of data **CIC-Darknet2020**, **IS-CXTor2016** to make sure we got a fair idea of how well it could sort things.

We used six models to sort the traffic: Logistic Regression, Support Vector Machine, Long Short-Term Memory, Convolutional Neural Network, CNN-LSTM and Transformer-LSTM. We did the steps to get the data ready for both sets so we could compare the models fairly. This way we could see how each model did under the conditions.

What we found out is that deep learning is better at figuring out traffic patterns than the way of doing things with machine learning. This is really true when the network traffic data has a lot of things that depend on each other over a time. The Transformer-LSTM model did a job because it combined the good things about Transformer and LSTM. This mix of the two is good for sorting through network traffic because it looks at how thingsre related and what happens one after the other.

Our framework gives us a way to try out different ways of sorting traffic. What we learned is important for people who study network traffic. It shows that using a mix of deep learning techniques can be really helpful. Maybe someday we can add data to our framework look into using simpler models that can work in real time and try out new ways of combining models or paying attention to certain things to make our sorting even better. The network traffic analysis field can use our findings to make things better. We can use the **CIC-Darknet2020**, **ISCXTor2016** datasets to try new things and make our framework even more useful, for network traffic classification.

XI. FUTURE WORK

The framework we have talked about is good at using machine learning and deep learning models for classifying network traffic.. There are still some things that we can do to make it better.

One thing we can do is add datasets to test the framework. These datasets should have kinds of network traffic and be from recent times. This will help us see how well the framework works with new and changing network environments. We should test it with datasets that have cyber threats, encrypted traffic and Internet of Things communications.

We can also try using deep learning models to make the framework better. We can use models like the Transformer and attention-driven hybrid networks to improve how accurate the framework is. We can also use learning to combine the results of many classifiers. This will help make the framework more robust and reduce errors.

Another thing we need to do is make the deep learning models use computing power. We can do this by making the models smaller and using power. This will allow us to use the framework in places where computing power's limited, like in edge devices and embedded security systems.

Now the framework only works with preprocessed datasets and does not work in real-time. We can change this by adding streaming data processing and online learning. This will allow the framework to classify network traffic as it happens.

We can also add techniques to make the framework more explainable. This means that the framework will be able to tell us why it made decisions. This will help people trust the framework more. Will help cybersecurity analysts understand what is going on.

Finally we can use the framework, in real-world cybersecurity applications. We can add it to intrusion detection systems, network monitoring platforms and intelligent security management solutions. This will allow the framework to help keep networks by classifying traffic accurately and automatically.

REFERENCES

- [1] A. Author, "Machine Learning for Network Traffic Classification," IEEE Journal, 2023.
- [2] B. Author, "Deep Learning Approaches for Encrypted Traffic," IEEE Conference, 2024.
- [3] A. Azab, M. Khasawneh, S. Alrabaee, K.-K. R. Choo, and M. Sarsour, "Network Traffic Classification: Techniques, Datasets, and Challenges."
- [4] C. Afifi, "Machine Learning With Computer Networks: Techniques, Datasets, and Models," *IEEE Access*, vol. 12, 2024.
- [5] D. Zhang, X. Chen, and Y. Xiang, "Robust Network Traffic Classification," **IEEE/ACM Transactions on Networking**. "Towards the Deployment of Machine Learning Solutions in Network Traffic Classification: A Systematic Survey," *IEEE Communications Surveys and Tutorials*.
- [6] E. Bayat and W. Jackson, "Deep Learning for Network Traffic Classification."
- [7] F. "Traffic Classification," *Information Fusion*, vol. 72, pp. 22–47, 2021.
- [8] G. "Dark Web Network Traffic Classification," **International Journal of Information Technology**, 2025.
- [9] H. "Paper A1," 2025.
- [10] I.CIC-Darknet2020 Dataset, Canadian Institute for Cybersecurity (CIC), University of New Brunswick.
- [11] J.ISCXTor2016 Dataset, Canadian Institute for Cybersecurity (CIC), University of New Brunswick.
- [12] F. Wang, Y. Liu, Z. Qin, and F. Dong, "A transformer-enhanced LSTM framework for robust malicious traffic detection in industrial control systems," *Knowledge-Based Systems*, vol. 321, art. no. 113725, Aug. 2025, doi: 10.1016/j.knosys.2025.113725.
- [13] N. C. Luong *et al.*, "Transformer-enhanced reinforcement learning: Fundamentals and applications in communication networks," *arXiv preprint arXiv:2606.05208*, May 2026.
- [14] S. A. Hosseini Seno *et al.*, "Graph and transformer-based deep learning paradigms for DDoS detection: A systematic and critical survey," *Computers, Materials & Continua*, vol. 88, no. 1, p. 1, 2026, doi: 10.32604/cmc.2026.078546.
- [15] P. Hemalatha, R. Rajesh, R. Vijayabharathi, K. Chandra Sekhar, J. Raja, and M. Ravichandran, "SecDL-Fuse: A hybrid secure deep learning framework for industrial cybersecurity using transformer-enhanced CNN and LSTM-GRU fusion," in *Proc. 5th Int. Conf. Computing and Communication Networks (ICCCN)*, 2026, pp. 377–386, doi: 10.1007/978-3-032-18910-3_34.
- [16] S. Ansari, K. A. Alnajjar, S. Majzoub, E. Almajali, A. Jarndal, T. Bonny, and S. Mahmoud, "Attention-enhanced hybrid automatic modulation classification for advanced wireless communication systems: A deep learning-transformer framework," *IEEE Access*, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3580574.
- [17] D. Choi, Y. Kim, C. Lee, and K. Sohn, "Dual-stream time-series transformer-based encrypted traffic data augmentation framework," *Applied Sciences*, vol. 15, no. 18, art. no. 9879, Sep. 2025, doi: 10.3390/app15189879.
- [18] Z. Zhao *et al.*, "Transformer-enhanced adaptive graph convolutional network for traffic flow prediction," *ACM Trans. Intell. Syst. Technol.*, 2026, doi: 10.1145/3729244.
- [19] S. B. Sabale and V. N. Khedkar, "A comprehensive survey of transformer-based models for video anomaly detection in surveillance systems," *International Journal of Safety and Security Engineering*, vol. 15, no. 10, p. 2143, 2025, doi: 10.18280/ijss.151017.
- [20] J. Zhang, Y. Xiang, Y. Wang, W. Zhou, Y. Xiang, and Y. Guan, "Network traffic classification using correlation information," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, no. 1, pp. 104–117, Jan. 2013.
- [21] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1988–2014, 2nd Quart. 2019.
- [22] A. W. Moore and D. Zuev, "Internet traffic classification using Bayesian analysis techniques," in *Proc. ACM SIGMETRICS Int. Conf. Measurement and Modeling of Computer Systems*, Banff, AB, Canada, Jun. 2005, pp. 50–60.
- [23] S. Rezaei and X. Liu, "Multitask learning for network traffic classification," in *Proc. 29th Int. Conf. Computer Communications and Networks (ICCCN)*, Honolulu, HI, USA, Aug. 2020, pp. 1–9.
- [24] J. Zhang, C. Chen, Y. Xiang, W. Zhou, and A. V. Vasilakos, "An effective network traffic classification method with unknown flow detection," *IEEE Trans. Netw. Service Manag.*, vol. 10, no. 2, pp. 133–147, Jun. 2013.
- [25] S. Bagui, X. Fang, E. Kalaimannan, S. C. Bagui, and J. Sheehan, "Comparison of machine-learning algorithms for classification of VPN network traffic flow using time-related features," *Journal of Cyber Security Technology*, vol. 1, no. 2, pp. 108–126, 2017.
- [26] M. Naas and J. Fesl, "A novel dataset for encrypted virtual private network traffic analysis," *Data in Brief*, vol. 47, art. no. 108945, Apr. 2023, doi: 10.1016/j.dib.2023.108945.

- [27] "Trust determination-based cybersecurity in cognitive radio networks using transformer-enabled residual bidirectional RNN," in *Proc. IEEE Conf.*, 2025, doi: 10.1109/ICCSP65057.2025.11085168.
- [28] R. Patel, I. A. Kimsanovich, V. Waiker, E. Muniyandy, S. M. Naidu, N. Mahamatov, and O. R. Shahin, "Transformer driven multi-agent reinforcement learning framework for integrated waste classification, forecasting and adaptive routing," *International Journal of Advanced Computer Science and Applications*, vol. 16, no. 11, p. 743, 2025, doi: 10.14569/ijacsa.2025.0161174.
- [29] J. Wang, N. Huang, H. Zhang, L. Liu, Q. Fu, K. Cao, X. Guo, and H. Jung, "Self-learning model fusion for network anomaly detection: A hybrid CNN-LSTM-transformer framework," *PLOS ONE*, vol. 20, no. 10, e0332502, Oct. 2025, doi: 10.1371/journal.pone.0332502.
- [30] V. Govindarajan and J. H. Muzamal, "Advanced cloud intrusion detection framework using graph based features, transformers and contrastive learning," *Scientific Reports*, vol. 15, art. no. 21487, Jul. 2025, doi: 10.1038/s41598-025-07956-w.
- [31] S. A. Wahab, S. Sultana, N. Tariq, M. Mujahid, J. A. Khan, and A. Mylonas, "A multi-class intrusion detection system for DDoS attacks in IoT networks using deep learning and transformers," *Sensors*, vol. 25, no. 15, art. no. 4845, Aug. 2025, doi: 10.3390/s25154845.
- [32] Y. Zhang, L. Zhang, and X. Zheng, "Enhanced intrusion detection for ICS using MS1DCNN and transformer to tackle data imbalance," *Sensors*, vol. 24, no. 24, art. no. 7883, Dec. 2024, doi: 10.3390/s24247883.
- [33] W. Du, J. Xue, X. Yang, W. Guo, D. Gu, and W. Han, "Transfformer: A novel Transformer-based framework to generate evasive malicious traffic," *Knowledge-Based Systems*, vol. 319, art. no. 113546, Jun. 2025, doi: 10.1016/j.knosys.2025.113546.
- [34] Z. Wu, H. Zhang, P. Wang, and Z. Sun, "RTIDS: A robust transformer-based approach for intrusion detection system," *IEEE Access*, vol. 10, pp. 64375–64387, Jun. 2022, doi: 10.1109/ACCESS.2022.3182333.
- [35] Z. Long, H. Yan, G. Shen, X. Zhang, H. He, and L. Cheng, "A transformer-based network intrusion detection approach for cloud security," *Journal of Cloud Computing*, vol. 13, art. no. 5, 2024, doi: 10.1186/s13677-023-00574-9.
- [36] H. Wang and W. Li, "DDosTC: A transformer-based network attack detection hybrid mechanism in SDN," *Sensors*, vol. 21, no. 15, art. no. 5047, Jul. 2021, doi: 10.3390/s21155047.
- [37] P. Appiahene, S. O. Berchie, E. Botchway, M. J. Ayitey, J. K. Dawson, H. N.-A. Mettle, S. Afrifa, and B. Tasci, "Network intrusion detection using a hybrid graph-based convolutional network and transformer architecture," *PLOS ONE*, vol. 21, no. 1, e0340997, Jan. 2026, doi: 10.1371/journal.pone.0340997.
- [38] Y. Liu and L. Wu, "Intrusion detection model based on improved transformer," *Applied Sciences*, vol. 13, no. 10, art. no. 6251, May 2023, doi: 10.3390/app13106251.
- [39] M. Aly and N. M. Alotaibi, "SHAP enhanced transformer GWO boosting model for transparent and robust anomaly detection in IIoT environments," *Scientific Reports*, vol. 15, art. no. 41037, Nov. 2025, doi: 10.1038/s41598-025-25033-0.
- [40] H. Wang and W. Li, "DDosTC: A transformer-based network attack detection hybrid mechanism in SDN," *Sensors*, vol. 21, no. 15, art. no. 5047, Jul. 2021, doi: 10.3390/s21155047.